ELI — Perception

ELI v2.5.34

November 2026

Contents

ELI Perception — Vision, Voice, Wake Word, Tone, OS Control **1**

Files . . . 1
Vision (vision.py + ambient_vision.py + analyze_image.py) . . . 3
Gaze tracking (gaze_engine.py) . . . 3
Screen locate & click (screen_locator.py + ui_ground.py) . . . 3
Deep screen analysis (screen_analysis.py + SCREEN_READ_ANALYZE) . . . 3
Cross-OS desktop input (desktop_capabilities.py + platform_compat.py) . . . 3
Speech-to-text (audio_stt.py, local_whisper_stt.py, mic_resolver.py) . . . 4
Wake word (wakeword.py, 2026-06-08) — self-trained, robust over music . . . 4
Voice profile + tone (voice_profile.py, 2026-06-08) — foundation for emotion . . . 5
Tone / emotion adaptor (cognition/tone_adaptor.py + cognition/emotion_palette.py) . . . 5
Animated face liveness (gui/widgets/eli_face.py + cognition/expression_state.py) . . . 5
Emotional memory + proactive check-in (cognition/emotion_timeline.py, v2.1.31) . . . 6
Text-to-speech (tts_router.py) . . . 6
OS control (os_controller.py) . . . 7
File analysers . . . 7
Honest assessment . . . 8
Update — 2026-06-09 (VRAM-aware STT; drag-drop keeps the full path) . . . 8

ELI Perception — Vision, Voice, Wake Word, Tone, OS Control

eli/perception/ (~20 files). ELI's senses and hands: local vision, speech-to-text, text-to-speech, a **self-trained wake-word detector**, a **voice-profile/tone** subsystem, and OS control. All local, no APIs, no third-party accounts.

Files

File	LOC	Role
mic_resolver.py	~580	**cross-platform mic auto-resolve** — ranked PortAudio candidates, isolated live probes, Linux PULSE_SOURCE pin
audio_stt.py	~1.6k	STT + mic capture + ducking + adaptive pause + wake/voice capture

File	LOC	Role
`wakeword.py`	~450	**self-trained, music-robust wake-word detector** (openWakeWord features + custom head)
`voice_profile.py`	~420	**prosody + labelled-emotion** (tone/question detection)
`tts_router.py`	1219	Piper/pyttsx3/espeak router + `char:/clone:/natural:` voice resolution; `neural_fallback_state()` makes a neural→Piper fallback observable
`voice_fx.py`	252	**voice styles** (calm / robotic / energetic / synthetic / refined) — base voice + ffmpeg effect chain; user preset store
`tts_xtts.py`	394	**voice cloning** from a reference sample (Coqui XTTS-v2). Zero-shot: `add_clone()` only registers a reference clip, conditioning happens at synthesis. Bundled in the Linux AppImage from v2.1.65; when absent the voice still registers and synthesis falls back to Piper — **loudly**, not silently, which was a live fault
`vision.py`	698	
`os_controller.py`	572	
`screen_locator.py`	497	locate UI elements on screen (AT-SPI → local VL-ground → OCR)
`screen_analysis.py`	~180	depth modes, prior-screen memory, research-context prompts
`ui_ground.py`	~320	local GGUF precision click + computer-use agent (no HTTP)
`desktop_capabilities.py`	~170	cross-OS input/screenshot/locate probe; feeds the awareness briefing and SELF_TEST
`media_deps.py`	~140	bundled-python + PATH discovery for yt-dlp/mpv/desktop CLIs
`gaze_engine.py`	358	webcam gaze/eye tracking daemon — see below
`log_rotation.py`	224	log housekeeping
`analyze_pdfs/image/mesh/csv.py`	~600	file-type analysers
`ambient_vision.py`	238	periodic screen glances (off by default)
`local_whisper_stt.py`	335	
`voice_worker_streaming.py`	small	streaming voice worker

Vision (`vision.py + ambient_vision.py + analyze_image.py`)

The working local-vision stack (see memory `eli-image-analysis`): - VL model (default Qwen2.5-VL, configurable) loaded via `Qwen25VLChatHandler`; **CPU clip forced** by monkeypatching `mtmd_cpp.mtmd_init_from_file(use_gp` — the GPU mtmd clip path **segfaults** on compute-7.5 cards. - **Hot-swap**: unload text model → load VL → infer → restore in `finally`, holding `gguf_inference._LLM_CALL_LOCK`. - **Co-resident fast path**: a small model (Moondream2 Q4) loaded first, with the text model's ctx capped (`vision_coresident_text_ctx`) so both fit 8GB; `prefer_fast=True` uses it without a swap. - Images downscaled to `vision_max_image_px` (1280) to avoid context overflow; `repeat_penalty/top_k/top_p` set to kill a repetition loop. - `ambient_vision.py`: optional periodic screen glances (OFF by default). A guarded daemon re-reads the toggle/interval each cycle and **skips a glance whenever the shared LLM lock is busy** (so it never steals the model mid-reply). Stores a short description as memory for rolling awareness.

Gaze tracking (`gaze_engine.py`)

A background thread: opens the webcam, runs real-time gaze estimation (MediaPipe iris landmarks, with a Haar-cascade fallback), applies a ridge-regression calibration model if a calibration file exists, smooths the output through a "OneEuroLikeFilter" (the 1€ Filter — a real HCI signal-smoothing technique built for exactly this: low-latency, jitter-free tracking of noisy human input), and writes `latest_gaze.json` every ~100ms. With no calibration it still runs, producing raw face-centre coordinates (useful for "is someone present" detection). Feeds a **dwell-click accessibility feature** — rest your gaze on a target for ~1s to click it, no mouse or voice needed (`GAZE_ENABLE`'s `dwell` argument). Settings key `gaze_engine_enabled`.

Screen locate & click (`screen_locator.py + ui_ground.py`)

Three-strategy chain for "find/click X on screen" — all local:

1. **AT-SPI** (Linux) — real widget tree: role, bounds, invoke action. Best for labelled buttons and accessible apps.
2. **Local VL-ground** (optional) — `ui_ground_backend=auto|local_gguf` runs a local GGUF (Phi-Ground / OS-Atlas / UGround naming in `models/`) or the primary VL with a JSON box prompt. For icon-only or non-text UI.
3. **OCR** — Tesseract word boxes + fuzzy match; click centre coordinate.

Computer-use agent (`computer_use_backend=auto|local_gguf`) — optional UI-TARS-style GGUF in `models/` for multi-step screenshot→action loops. Off by default.

Configure on GUI **Screen tab** or `settings.json`: `ui_ground_backend`, `computer_use_backend`, `screen_analysis_depth` (`quick|standard|deep`).

Deep screen analysis (`screen_analysis.py + SCREEN_READ_ANALYZE`)

Voice examples that route to a real screenshot + VL + OCR (+ text-model fusion):

- "What's on my screen?" — standard depth
- "Analyze my screen in depth / exactly what's on my screen" — deep audit
- "How does this relate to my research?" — pulls local memory + KG context
- "Do you remember seeing this in previous sessions?" — prior `screen_awareness` glances

No network. Missing OS tools (mpv, tesseract, ydotool) are offered via `grounded_remediation` **after** a failed attempt — never as a pre-install gate.

Cross-OS desktop input (`desktop_capabilities.py + platform_compat.py`)

OS	Mouse/keyboard	Screenshot
Linux X11	xdotool → PyAutoGUI	scrot

OS	Mouse/keyboard	Screenshot
Linux Wayland	ydotool+ydotoold → xdotool (XWayland) → PyAutoGUI	grim+slurp
Windows	PyAutoGUI	Pillow ImageGrab
macOS	PyAutoGUI (+ Accessibility permission)	screencapture
Android/Termux	intents only	N/A

See `docs/CROSS_PLATFORM.md` for the full matrix and install commands per OS.

Speech-to-text (`audio_stt.py`, `local_whisper_stt.py`, `mic_resolver.py`)

Microphone selection (`mic_resolver.py`, v2.3.44). On every OS, ELI builds a ranked list of PortAudio inputs — USB and headset/Bluetooth (AirPods, Turtle Beach Chat, Trust USB, …) before built-in, webcam, and virtual loopback routes — and probes each in an isolated subprocess at the device's native sample rate until one delivers live audio. Linux additionally tries PipeWire/Pulse sources with a process-local `PULSE_SOURCE` pin after direct hardware. Honours `ELI_MIC_DEVICE_INDEX` (explicit override) and `ELI_MIC_AUTORESOLVE=0` (OS default only). Diagnostic: `python -m eli.tools.mic_diag`.

A large, feature-dense STT module: - Mic capture + Whisper transcription (`local_whisper_stt`). - **Output ducking** — lowers system sink volume while listening (`_eli_duck_output`/`_eli_restore_output` via `wpctl`) so ELI doesn't hear itself. - **Echo/self-hearing suppression** (`_eli_echo_like_assistant_output`, `_eli_media_probably_audible`). The voice (`tts_router._run_tts`, every engine) holds the speaking lock for the whole reply, holding its process id; the mic treats it as stale only when that process has ended (`speaking_lock_held`). Taken as stale after 30 seconds, a minute-long reply was heard and transcribed half-way through. - **Voice commands while a reply is coming**: a short one said after the wake word waits for the reply to finish (`_send_waiting_voice`) instead of being dropped; a long one still interrupts. - **Decoder hints and doubt** (`local_whisper_stt`): the wake words and the apps on the machine the person has named in conversation go to faster-whisper as `hotwords` ("open spotify" in a noisy room was heard as "old thing spot employee"). A command whose every segment is below the decoder's own failed-decode bar is not carried out: the window opens again and the status bar says it was not clear (`heard_clearly`). - **Playback** opens at the output device's own rate, the voice resampled to it, instead of failing first on a device that takes only its own rate. - Transcript cleanup: `_cleanup`, `_collapse_repeated_phrase`, `_eli_fast_command_alias` (maps spoken phrases to commands), `_is_safe_direct`/`_allow_direct_chat` (gates which utterances bypass to chat). - ALSA stderr suppression to keep the console clean. - **Duration-adaptive end-of-phrase pause** (`_listen_adaptive_pause`, 2026-06-08) — stock `sr.listen()` reads `pause_threshold` once per phrase, so one value can't be both snappy for commands and tolerant of long dictation. A faithful copy of sr's capture with a single dynamic condition: short commands finalise after `ELI_STT_SHORT_PAUSE` (0.5s) of silence; a prompt speaking past `ELI_STT_LONG_AFTER` (12s) needs `ELI_STT_LONG_PAUSE` (2s), so a mid-sentence pause no longer cuts it. Flag-gated (`ELI_STT_ADAPTIVE_PAUSE`) with fallback to stock `listen()`. - **Generic mic-capture script** — one mechanism (no second microphone) drives both wake-word enrollment and voice/emotion training: a list of `{prompt, sink}` steps, each captured (past the TTS-echo guards) clip routed to its sink, the next cue spoken, a `done` callback after the last. `begin_capture_script` / `begin_wake_enrollment` / `begin_voice_training`. - **Acoustic wake hook** — when unarmed and a wake model is trained, the captured audio is scored by `wakeword` and, if it fires, the wake word is injected so the existing `VoiceGate` arms — catching the wake word even when whisper transcribed the music. **Per-turn tone hook** — on a real command, `voice_profile.classify_tone` is run and published on a side-channel for cognition (below).

Wake word (`wakeword.py`, 2026-06-08) — self-trained, robust over music

Transcription-based wake matching can't hear "computer" over loud music (whisper transcribes the music). Instead ELI **trains its own** detector, 100% locally: - **Positives**: the wake phrase synthesised

by ELI's OWN **Piper TTS** across several voices and speeds. - **Augmentation (the robustness)**: each positive is mixed with noise/music at random SNRs, so the classifier learns to spot the wake word *through* a music bed. - **Features**: openWakeWord's open, bundled melspectrogram→embedding extractor (no account, no download barrier). - **Custom head**: a small torch classifier ELI owns; `train_model()` → `models/wakeword/eli_wake_head.pt` (gitignored). `WakeDetector.score_audio/is_wake` slide a 1.5s window. Validated: clean wake = 1.00, wake+loud music (3 dB) = 1.00, hard-negative/music-only = 0.00. - **User-settable phrase** — `get/set_wake_phrases` persists any phrase ("change the wake word to athena"); it feeds both this model and the transcription matcher. - **Personalisation** — enrolled real-mic clips of the user are folded in as heavily- weighted positives. - Actions: `WAKE_SET` / `WAKE_TRAIN` / `WAKE_ENROLL`. Fully fallback-safe (`ELI_WAKE_ACOUSTIC=0`; no model → the transcription matcher).

Voice profile + tone (`voice_profile.py`, 2026-06-08) — foundation for emotion

Deliberately SEPARATE from the wake word — this is *how* the user speaks, the basis for tone/emotion and question-vs-statement: - **Prosody (real, numpy only)**: per-clip autocorrelation **F0/pitch** track, energy, voiced ratio, speaking rate, and a **terminal-pitch slope** (`analyze_prosody`). - **Question vs statement** (working): rising terminal pitch ⇒ question (`question_or_statement`). - **Labelled emotion**: `add_labelled_sample` + a nearest-centroid classifier (`train_emotion_classifier`) over neutral/happy/angry/excited/sad — robust for the small data a quick enrollment yields, upgradeable to a NN without touching callers. - **`classify_tone`** returns emotion + confidence, arousal, and the question/statement cue. `build_profile` learns the user's baseline so tone is scored *relative* to how they normally sound. - **`TRAIN_VOICE`** runs one unified guided session (wake reps + the same line said in each emotion); modules stay separate, the user does one flow. - **Wired into cognition**: STT publishes the per-turn read on a fresh, timestamped side-channel (`set_last_tone`); `engine._build_enhanced_system` reads `get_last_tone` and prepends a speaker-voice cue so ELI adapts its delivery (warmer if upset, brisk if energetic). `ELI_VOICE_TONE=0` disables. Complementary to the text-based `cognition/tone_analyzer` (persona preferences), not a duplicate.

Tone / emotion adaptor (`cognition/tone_adaptor.py` + `cognition/emotion_palette.py`)

Decides the emotion ELI **expresses** and drives three output channels at once.

- **Two identifiers** (the "2 systems"): *acoustic* (`voice_profile.get_last_tone` — the user's voice this turn) and *semantic* (`detect_text_emotion` — the content of what they said). `_fuse()` combines them (agreement boosts confidence; voice slightly favoured).
- **Empathetic response policy**, not mimicry: a distressed user → `tender`, heat → `calm`, playful → `playful`. An explicit override (`set_tone("be comedic"/"talk street")` via `SET_TONE`/`CLEAR_TONE`) wins outright and persists.
- **Extensible palette** (`emotion_palette.py`): 20 built-in tones (comedic, professional, deadpan, street_smart, gremlin, tender, ...), each carrying a persona directive, Piper prosody profile, and avatar expression; users add their own in `config/voices/tones.json`.
- **Three output channels**: text (a directive folded into `_build_enhanced_system`), voice (`tts_router._piper_prosody` overlays the tone's pace/expressiveness/pauses **and `_apply_tone_pitch` shifts the actual audio pitch** so sad sounds lower, ecstatic higher — applied on the Piper, natural-neural and clone paths), and face (`world/avatar/persona_mapper` + the animated `gui/widgets/eli_face.py` take the expression).
- **Core personality is never overwritten** — the directive explicitly shades *how* ELI delivers, not *who* he is (emergent-voice principle). `ELI_TONE_ADAPT=0` disables.

Animated face liveness (`gui/widgets/eli_face.py` + `cognition/expression_state.py`)

The procedural face reacts in real time to what ELI is *doing*, via three cheap flags in `expression_state` (priority: thinking > speaking > tone): - **Lip-sync** — the mouth animates while TTS plays (`tts_router._run_tts` sets the `speaking` flag; amplitude-driven when available, else a natural talk oscillation). - **Thinking look** — a focused/reflective face while the model generates (the GUI sets

`thinking` around `is_generating`). - Otherwise it shows the current expressed tone. Pure-Qt, embedded in the world panel.

Emotional memory + proactive check-in (`cognition/emotion_timeline.py`, v2.1.31)

`tone_adaptor` shades the *current* reply and then forgets: the acoustic read lives in a 12-second slot, the semantic read is recomputed from the last utterance. That makes ELI **reactive**. The emotion timeline is the durable record that makes it **proactive**.

- **Persistence** — every fused read (neutral included, so the baseline isn't built only from bad turns) is appended to `emotion_events` in `user.sqlite3`, with the utterance that produced it, the arousal, and **the action ELI ran on the preceding turn**.
- **`assess()`** returns EVIDENCE, never a phrase: the sustained run and its dominant emotion, the `neutral→negative` transition, the ELI action at the turn the mood turned, and whether it is *unusual for this user* — measured against their own baseline, so a naturally reserved person is never read as upset.
- **Gates** (all must pass before ELI raises anything): a run of N reads rather than a single spike, per-read confidence above a floor, a credible baseline, and a cooldown so ELI cannot nag. `assess()` reports its `reason` either way, so a *non*-check-in is explainable too.
- **Two surfacing paths.** In-conversation, `evidence_block()` is folded into the system prompt and **the model decides whether and how to raise it** — the wording is ELI's, never a template (see [ELI No Hard-Coded Responses]). Unprompted, the proactive daemon runs the same assessment on its tick, LLM-synthesises an opener from the evidence, and pushes an `emotion_checkin` onto the suggestion queue (+ a pending proposal so a plain "yes" routes). With no resident model the daemon emits **nothing** rather than a canned line.
- **`trend_line()`** runs every turn regardless, giving ELI continuity on how the conversation has been feeling.
- **User control** — Settings ▸ Cognition ▸ *Emotional awareness*: master toggle, exchanges before a mood counts, minimum confidence, check-in cooldown, recent-mood window, baseline history. All read at call time (no restart), all defaulting to the shipped constants. A *Tone of voice* dropdown in the same tab pins the register ELI answers in (Auto + the 20 palette tones) via `tone_adaptor.set_tone`. Kill switch: `ELI_EMOTION_MEMORY=0`.

Text-to-speech (`tts_router.py`)

Multi-backend router with graceful fallback: **XTTS-v2 natural neural** (opt-in) → **Piper** (packaged binary under `tts_piper/` or `models/tts/piper`) → **pyttsx3** → **espeak-ng / espeak**. Resolves voices from several candidate dirs so a packaged build or a source checkout both work.

Natural neural voice (`natural:<id>`, `tts_xtts.py`). XTTS-v2 driven by its **built-in studio speakers — no clone/reference needed** — is the human-like, expressive, punctuation-aware path. `list_natural_voices()`/`synthesize_natural_wav()` are cheap to probe (never load the ~1.8GB model just to enumerate) and **fall back to Piper** when the neural extra isn't installed, so a `natural:` voice never goes silent. It is selectable at the **startup model picker / first-boot wizard** ("Voice engine: Piper vs Natural") — the choice is persisted *before* hardware tuning so its VRAM is a deliberate up-front decision (`ELI_XTTS_DEVICE` pins the device); low-VRAM boxes keep fast Piper. Enable with `pip install -e ".[natural]"` (from the ELI project root).

Expressive Piper prosody. Piper at bare defaults sounds flat and clips full stops. The router now passes tuned `--sentence_silence` / `--length_scale` / `--noise_scale` / `--noise_w` (`_piper_prosody_args()`, env-overridable, `ELI_TTS_PROSODY=0` to disable) so `?`/`!`/`.` land as intonation and pauses — the "Amy sounds flat" fix.

No robotic voices. `list_voices()` hides the espeak `sys:` voices and Piper `low`/`x_low` tiers by default (they sound synthetic), surfacing them only as a genuine last resort when the box has nothing better, or when `ELI_TTS_ALLOW_ROBOTIC=1`.

Voice library (`runtime/voice_assets.py`) — the shipped pack is a starting point, not the limit. The module fetches the upstream Piper index (**166 voices across 45 languages, 38 of them English** — US/GB, Scottish, Northern and Southern English) and exposes it through `list_available_voices()` with per-voice presence, download size and licence, cached on disk so the picker still works offline. `download_voice()` streams from the index's exact paths and **verifies the published md5** before the file is put in place, so a truncated or proxy-mangled download is rejected rather than surfacing later as a corrupt-model crash. The GUI path is Settings > VOICE / TTS > *Get more voices / accents…* (`gui/widgets/voice_downloader.py`); every voice dropdown repopulates on install.

Redistribution policy. `RESTRICTED_VOICE_NAMES` (ryan — CC-BY-NC-SA; lessac — Blizzard/Lessac, not cleared; cori — pending) are never put in a release asset. The rule is keyed on the **voice name, so every quality variant is covered**, and `scripts/asset_release_policy.py` imports it so the build and the runtime cannot drift. The user may still download them for personal use; the dialog states this rather than hiding the voice.

Config integrity. A `.onnx` without its `.onnx.json` cannot be loaded by Piper, so it is invisible to `list_voices()` while still occupying ~60 MB. `incomplete_voices()` finds these and `repair_voice_configs()` fetches just the missing ~5 KB config. `tests/test_voice_library.py` guards against shipping the broken state again.

Style presets (`voice_fx.py`, `char:calm`, `char:robotic`, …) are a redistributable Piper base voice plus an ffmpeg effect chain (pitch / speed / filters). They are **generic sound styles**, not bundled impersonations of copyrighted characters. Ideal Piper bases for some historical presets (lessac, joe, ryan) are excluded from the public bundle over dataset licensing; each preset carries an **ordered, gender-matched `fallback` chain** against installed voices only. **Voice cloning** (`tts_xtts.py`, `clone:<name>`) reproduces a voice from a short reference clip via Coqui XTTS-v2 — an opt-in extra (`eli-v2.0[clone]`) that falls back to a normal voice when the model isn't installed rather than going silent. Both `char:` and `clone:` resolve through `synthesize_wav` (browser voice) and `_run_tts` (host playback), and appear in the GUI voice picker.

OS control (`os_controller.py`)

Platform-aware (Linux/Windows/macOS) screenshot, volume, keyboard, clipboard. `take_screenshot(region)` picks the right tool per platform (`gnome-screenshot`, PIL `ImageGrab`, platform CLIs); fails gracefully where a capability isn't available (e.g. area screenshots on some Windows installs). `screen_locator.py` finds UI targets for click/automation.

File analysers

`analyze_pdfs`, `analyze_image`, `analyze_csv` — typed handlers behind the `ANALYZE_*` actions, feeding extracted content (text/OCR/structure) into the grounded pipeline.

Several folders or files named in one message all go to one job (`paths_named`, `_route_several_paths`); a path that is missing is reported; with nothing named and no work in hand ELI asks which, never the current folder. Asked to analyse, a folder is read in full, in pieces; asked to summarise, from the opening of each document, and the report says which. A summary that comes back empty is a failed job: the documents are indexed and nothing has been analysed.

A folder (`ANALYZE_PDF_FOLDER`, `eli/runtime/folder_analysis.py`) is read through the document index: every PDF is stored whole there, one summary covers all of them (in parts of what one call has always read, then joined), each call is told how many words its reply holds (the reply limit times the model's measured characters per token, over the characters per word of what it reads), a reply cut at its limit still ends at its last finished line and drops a heading left with nothing under it, and the result, with every document listed by its path in the folder and any unreadable files, is saved as a PDF (`eli/runtime/rich_document.py`: matplotlib typesets the formulas, Qt lays out the page, .docx where there is no window). The same rendering lays out chat replies and task results, so `$\phi$` reads as φ.

Formulas match the text: each is drawn at the size whose x stands as tall as the text font's x (Computer Modern's x is 0.44 of its size, a sans font's about 0.54, so at the same size formulas looked a size smaller),

and `put_html` stands each on the text's baseline with Qt's `AlignBaseline`, sizing the image format's font so its descent is the depth the formula was drawn with. Qt's HTML reader can only centre an image in the line box, which moves with whatever else the line holds, or stand its lower edge on the baseline, which lifts subscripts and fractions. A fraction standing in a displayed formula is full size (`\dfrac`), as in TeX. The PDF is laid out at the resolution text is measured in and drawn page by page in black: `QTextDocument.print_` re-lays an unpaged document out in printer units, where an image's baseline is still measured in screen pixels, and drawing with the window's palette printed ELI's dark-theme text near white. .docx pictures are lowered by their depth (`w:position`). It used to file each piece of every PDF as a personal memory and summarise from the first 17 documents.

Honest assessment

- **Strong:** this is the layer that makes ELI genuinely multimodal and local — eyes (VL + OCR + ambient), ears (Whisper + ducking), mouth (Piper/espeak), hands (cross-platform OS control). The vision co-residence + hot-swap + CPU-clip workaround is real engineering against hard 8GB-GPU constraints.
- **Weak / watch:**
  1. **Vision latency** — the hot-swap unloads/reloads the text model per glance; even the co-resident path runs CPU clip (~3.5s). Acceptable, not fast.
  2. `audio_stt.py` is a **2.5k-line** module mixing capture, ducking, echo suppression, command aliasing, and cleanup — wants splitting.
  3. **Residual "7B" comment** in `ambient_vision.py` ("the 7B vision model hot-swaps…") — cosmetic, but inconsistent with the model-agnostic line; should read "the vision model". (Flagged for a future scrub.)
  4. `gaze_engine.py` is experimental and off by default — fine, but it's carried weight.
  5. Platform tools are detected at call time; a machine missing `gnome-screenshot`/`wpctl`/`piper` degrades silently to fallbacks (good) but there's no single "what perception capabilities do I actually have here?" probe surfaced to the user.

Update — 2026-06-09 (VRAM-aware STT; drag-drop keeps the full path)

- **STT is VRAM-aware** (`local_whisper_stt`). faster-whisper defaults to GPU only when the card is big enough for whisper AND a typical main model (≥12 GB total VRAM), else CPU — so on an 8 GB card whisper no longer preloads ~2 GB and starves the main model (observed: free_vram 4083 MB → gpu_layers=11; after the fix free_vram 7092 MB → gpu_layers=99). `small.en` int8 on CPU is ~1–2 s for a short command. Explicit `ELI_WHISPER_DEVICE` still wins; `ELI_WHISPER_GPU_MIN_MB` tunes the threshold. *Superseded in 2.5.18:* no size threshold. Whisper goes on CUDA device 0 only when that card still holds its measured footprint after the language model's load plan (see `inference_and_hardware.md`, STT VRAM awareness); `ELI_WHISPER_GPU_MIN_MB` is gone.
- **Drag-and-drop keeps the FULL path.** A file dropped into the chat input now expands to `[File: <full path>]` followed by the inlined content (was content + basename only), so "fix/examine/edit this file" can resolve the file on disk while "summarise this" still has the content. PDFs keep their path too; combined with the FIX_FILE last-file memory, a bare follow-up "fix it" recovers the file named a turn earlier.